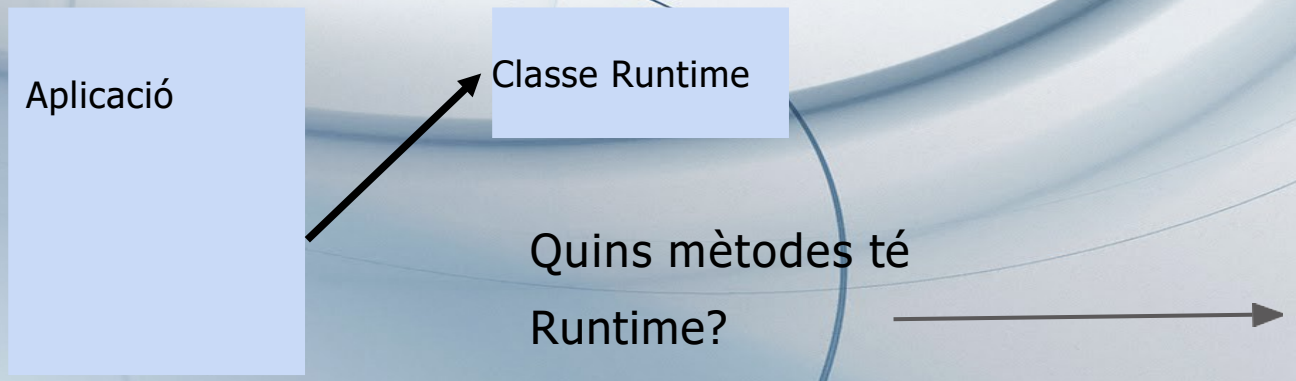
Tema 2:
Programació multifil

Setmana 1. Creació de processos en Java

Process i Runtime

Java disposa de diverses classes per a la gestió de processos.

Cada aplicació disposa d'una instància Runtime que representa entorn de l'aplicació.



Classe runtime: Process i Runtime

Classe runtime 2 mètodes importants:

- *static Runtime getRuntime():* retorna l'objecte Runtime associat a l'aplicació en curs.
- *Process exec (String comanda):* executa l'ordre especificada a la comanda en un procés separat. Retorna objecte process que es pot utilitzar per controlar interacció del programa java amb el nou procés.

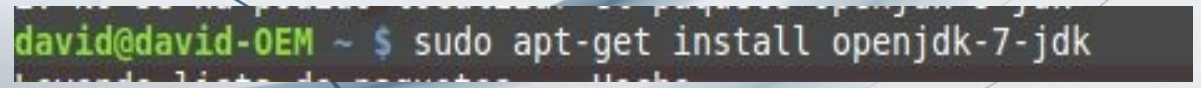
Process i Runtime

Fem que un procés execute una aplicació, per provar els mètodes descrits.

Instal·lació compilador Java Windows

Instal·lar jdk Java. enllaç.

Instal·lació compilador Java Linux



Instal·lació en Windows

Si a l'instal·lar java, t'apareix un missatge que no reconeix el comando javac, és perquè el directori java/bin no està afegit al PATH (variable del sistema, no de l'usuari). Posa'l al principi.

Exemple 1

Mostra com executar una aplicació:

- A Windows executem Notepad.
- A Linux executem gedit.

Exemple: executar notepad

Exemple1.java

```
1
2   public class Exemple1{
3   public static void main(String[] args) {
4       Runtime r=Runtime.getRuntime();
5       String comando="gedit"; //A Windows canviar per "notepad"
6       Process p;
7
8       try{
9           p=r.exec(comando);
10          } catch(Exception e) {
11              System.out.println ("Error en "+comando);
12              e.printStackTrace();
13              }
14      }
15
16  }//Exemple1
17
```

obtinc objecte Runtime

Variable tipus procés

Execute comandament

Compile i execute
(És el mateix en Windows i Linux)

```
C:\pss>javac Ejemplo1.java
C:\pss>java Ejemplo1
```

Exemple: executar notepad

Què passa amb ordres Windows o Linux no tenen executable?

→ Utilitzem *cmd /c dir* (Windows) o *ls* (Linux)

Modifiquem en Exemple1 *string comandament = "CMD /C DIR" o "ls"*

```
C:\pss>javac Ejemplo1dir.java
C:\pss>java Ejemplo1dir
```

→ No obtinc eixida PER QUÈ?

Classe Process

Per llegir la eixida necessitem usar l'objecte Process, que s'obté d'aquesta manera:

p=r.exec(comando);

Classe Process

Classe process inclou el mètode **getInputStream()** que ens permet llegir el fluxe o stream d'eixida del procés, és a dir el que la comanda va escriure a la consola d'eixida.

Anem a vore com s'implementaria →

Process. Desar stream eixida

```
p = r.exec(comandament);
```

execute comandament

```
InputStream is = p.getInputStream();
```

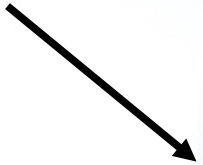
guarde el flux d'eixida en la variable «is» de tipus InputStream

```
BufferedReader br = new BufferedReader (new InputStreamReader(is));
```

guarde en la variable «br» de tipus BufferedReader per després treure línia a línia

Process. Mostra flux eixida

```
String linia;
while ((linia=br.readLine())!=null) //llegeix una línia
System.out.println (linia);
```



Llegeix línia a línia de br i va mostrant amb System.out.println (línia)

Process. Mostra fluxe eixida

Mentre s'executa subprocés (*cmd /c dir*),he de fer que el procés actual espere.

El mètode **waitFor()** fa que el procés actual espereu fins que el subprocés representat pel **Process** finalitze.

Retorna 0 si ha finalitzat correctament.

Exemple2

Exemple2.java

```java
import java.io.*;
public class Exemple2{
public static void main(String[] args) {
    Runtime r=Runtime.getRuntime();
    String comando="ls";
//  String comando="cmd /c dir"; //Windows
    Process p=null;

    try{
        p=r.exec(comando);
        InputStream is=p.getInputStream();
        BufferedReader br=new BufferedReader(new InputStreamReader (is));
        String linea;
        while ((linea=br.readLine())!=null)//llegig una línia
            System.out.println(linea);
        br.close();
        } catch(Exception e) {
            System.out.println ("Error en "+comando);
            e.printStackTrace();
            }

    //comprobació d'error: 0 bé - 1 mal
    int exitVal;
    try {
        exitVal=p.waitFor();
        System.out.println("Valor de salida "+exitVal);
        } catch(InterruptedException e){
            e.printStackTrace();
            }
    } //del main
}//Exemple2
```

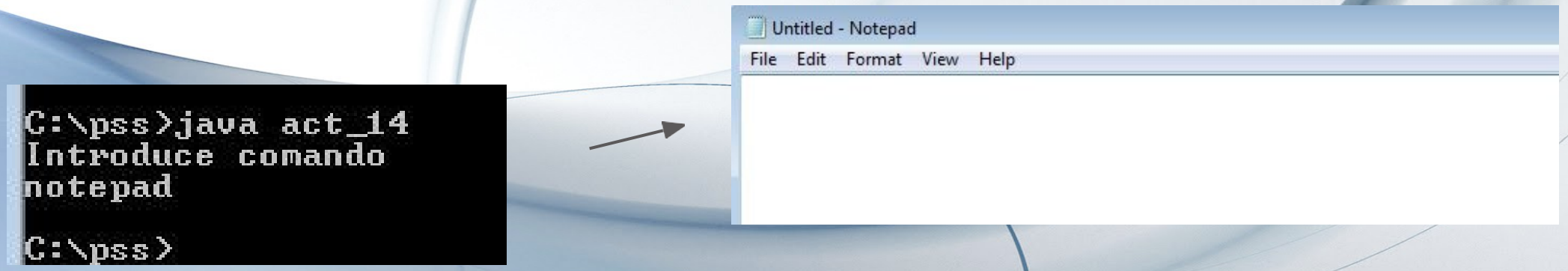
geany_run_script_6YAPA1.sh

```
08 - ut 2 Programación Multihilo . Métodos sincronizados.pdf
09 - ut2.Programación Multihilo. Productor y Consumidor. Sincronización de hilos
.es.ca.odp
09 - ut2.Programación Multihilo. Productor y Consumidor. Sincronización de hilos
.es.ca.pdf
09 - ut2.Programación Multihilo. Productor y Consumidor. Sincronización de hilos
.es.ca.pptx
09 - ut2.Programación Multihilo. Productor y Consumidor. Sincronización de hilos
.pdf
12 - Práctica Cuenta2.es.ca.docx
12 - Práctica Cuenta2.es.ca.pdf
12 - Práctica Cuenta2.pdf
Ejemplo1.class
Exemple1.class
Exemple1.java
Exemple2.class
Exemple2.java
Valor de salida 0


------------------
(program exited with code: 0)
Press return to continue
```

Activitat T2S1P1Quasibash

Realitza un programa Java usant Runtime i Process que reba des de la línia de comandos un nom de comando i l'execute.



Process. getErrorStream()

Què passa si es produeix error al llançar el procés?

Process posseeix el mètode getErrorStream() que obté el flux amb els possibles errors.

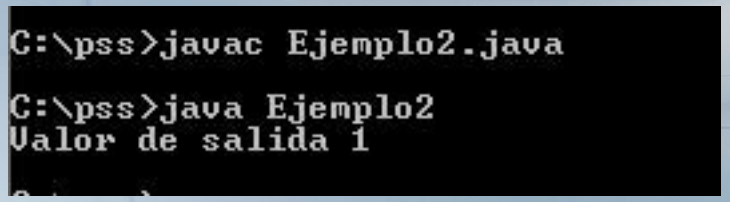
Process. getErrorStream()

Mètode **getErrorStream().**

Ens permet llegir possibles errors que es produeixen al llançar el procés.

Edita l'Exemple2.java i modifica la comanda per "cmd /c dir" o "ls" per provocar error.

L'eixida és:



Process. getErrorStream()

Si afegim el següent codi a exemple2:

flux d'errors de process

```
try {
    InputStream er=p.getErrorStream();
    BufferedReader brer=new BufferedReader(new InputStreamReader(er));
    String liner=null;
    while ((liner=brer.readLine())!=null)
        System.out.println ("ERROR >"+ liner);
    }catch (IOException ioe){ ioe.printStackTrace();     }
```

guarde en buffer i llegisc línia a línia

Process. getErrorStream()

Modifiquem l'ordre per provocar error

```
String comando="CMD /C DIRR"; //windows
```

Obtenim la següent eixida:

```
C:\pss>java Ejemplo2
Valor de salida 1
ERROR >'dirr' is not recognized as an internal or external command,
ERROR >operable program or batch file.
```

Linux: *Cal que modifiqueu la variable "comando" per a que es provoque l'error. Heu de tindre en compre que l'error que es captura és de l'execució del comando.*

Redireccionant a fitxers

Anem a vore si és possible redireccionar la eixida a un fitxer.

Process. FileOutputStream i PrintWriter

Creem un programa Java que pinta 5 vegades la salutació que se li introdueix per línia ordres

Unasalutacio.java

```
1  public class Unasalutacio {
2      public static void main(String[] args){
3      if (args.length<1){
4          System.out.println ("Cal una salutació...");
5          System.exit(1);
6          }
7      for (int i=0;i<5;i++){
8          System.out.println(i+1 + "."+ args[0]);
9          }
10     }//fi main
11 } //unasalutacio
12
```

Process. FileOutputStream i PrintWriter

Creem un programa Java que pinta 5 vegades la salutació que se li introdueix per línia ordres

```
[ramon@nowhere Exemples]$ java Unasalutacio "HOLA MÓN"
1.HOLA MÓN
2.HOLA MÓN
3.HOLA MÓN
4.HOLA MÓN
5.HOLA MÓN
[ramon@nowhere Exemples]$ java Unasalutacio "HOLA MÓN" > fitxer.txt
[ramon@nowhere Exemples]$ cat fitxer.txt
1.HOLA MÓN
2.HOLA MÓN
3.HOLA MÓN
4.HOLA MÓN
5.HOLA MÓN
```



Es crea fitxer amb eixida

Process. FileOutputStream i PrintWriter

Ara anem a intentar cridar a "Unasalutacio" des del programa "Exemple2".

Caldrà modificar el comandament i introduir "Unasalutacio" i el redireccionament a un fitxer.

Process. FileOutputStream i PrintWriter

Canviem en exemple2 la comanda i escrivim:

```
3   public static void main(String[] args) {
4       Runtime r=Runtime.getRuntime();
5       String comando="java Unasalutacio \"Hola Món!!\">exemple.txt";
6   //  String comando="ls";
7   //  String comando="cmd /c dir"; //Windows
8       Process p=null;
```

```
C:\pss>javac Ejemplo2.java

C:\pss>java Ejemplo2
1.Hola Mundo!!>ejem.txt
2.Hola Mundo!!>ejem.txt
3.Hola Mundo!!>ejem.txt
4.Hola Mundo!!>ejem.txt
5.Hola Mundo!!>ejem.txt
Valor de salida 0
```

Compilem i executem

No genera fitxer d'eixida

Per què?

```
[ramon@nowhere Exemples]$ javac Exemple2.java
[ramon@nowhere Exemples]$ java Exemple2
1."Hola
2."Hola
3."Hola
4."Hola
5."Hola
Valor de salida 0
```

Process. FileOutputStream i PrintWriter

exec() no actua com a intèrpret de comandaments o shell, només executa programes.

Si volem redirigir a un fitxer haurem d'afegir-ho per programació mitjançant classes:

- FileOutputStream
- PrintWriter

Exemple3. FileOutputStream i PrintWriter

Codi 1/3

NOTA: 2 coses a corregir!

Exemple3.java

```
1   import java.io.*;
2   public class Exemple3 {
3       public static void main(String[] args) {
4           Runtime r=Runtime.getRuntime(); //objecte runtime
5           String comando="java Unasalutacio \"Hola Món!!\" "; //comando a executar i params
6           Process p=null; //inicialitze vble process
7
8           //control params entrada
9           if (args.length<1){
10              System.out.println("Se necesita nombre de fichero..");
11              System.exit(1);
12          }//fi control parametros
```

Exemple3. FileOutputStream i PrintWriter

codi 2/3

Creem objecte FileOutputStream i PrintWriter

Imprimisc en fitxer

```
13
14      try {
15          //fitxer al que s'enviará l'eixida del programa Unasalutacio
16          FileOutputStream fos=new FileOutputStream(args[0]);
17          PrintWriter pw=new PrintWriter(fos);
18
19          //execute comando
20          p=r.exec(comando);
21
22          //cree buffer
23          InputStream is=p.getInputStream();
24          BufferedReader br=new BufferedReader(new InputStreamReader (is));
25          String linia;
26          //llegisc línia a línia i imprimisc en el fitxer
27          while ((linia=br.readLine())!=null)//llegix una línia
28              {
29              System.out.println("Es necessita nom de fitxer..");
30              System.out.println("Inserisc en "+args[0]+" > "+linia);
31              pw.println(linia); //la inserisc en el fitxer
32              }//fi bucle línies
33          br.close();
34          pw.close();
35          }
36          catch (Exception e) {e.printStackTrace();}
37
```

Exemple3. FileOutPutStream i PrintWriter

codi 3/3

```
37
38          //comprobació d'errors: 0 bé - 1 malament
39          int exitVal;
40          try {
41              exitVal=p.waitFor();
42              System.out.println("Valor d'eixida "+exitVal);
43              } catch(InterruptedException e){
44                  e.printStackTrace();
45                  }
46
47          }//fi main
48  }//fi Exemple3
49
```

Exemple3. FileOutputStream i PrintWriter

```
C:\Users\david\Desktop\procesos_java1>javac Ejemplo3.java

C:\Users\david\Desktop\procesos_java1>java Ejemplo3 fic.txt
Inserto en fic.txt > 1.Hola Mundo!!
Inserto en fic.txt > 2.Hola Mundo!!
Inserto en fic.txt > 3.Hola Mundo!!
Inserto en fic.txt > 4.Hola Mundo!!
Inserto en fic.txt > 5.Hola Mundo!!
Valor de salida 0
```

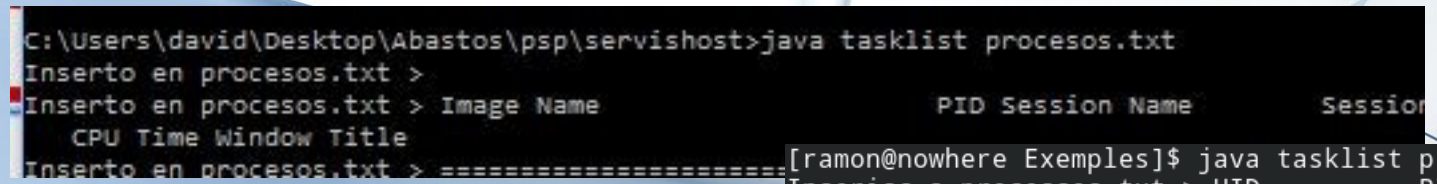
```
C:\Users\david\Desktop\procesos_java1>more fic.txt
1.Hola Mundo!!
2.Hola Mundo!!
3.Hola Mundo!!
4.Hola Mundo!!
5.Hola Mundo!!
```

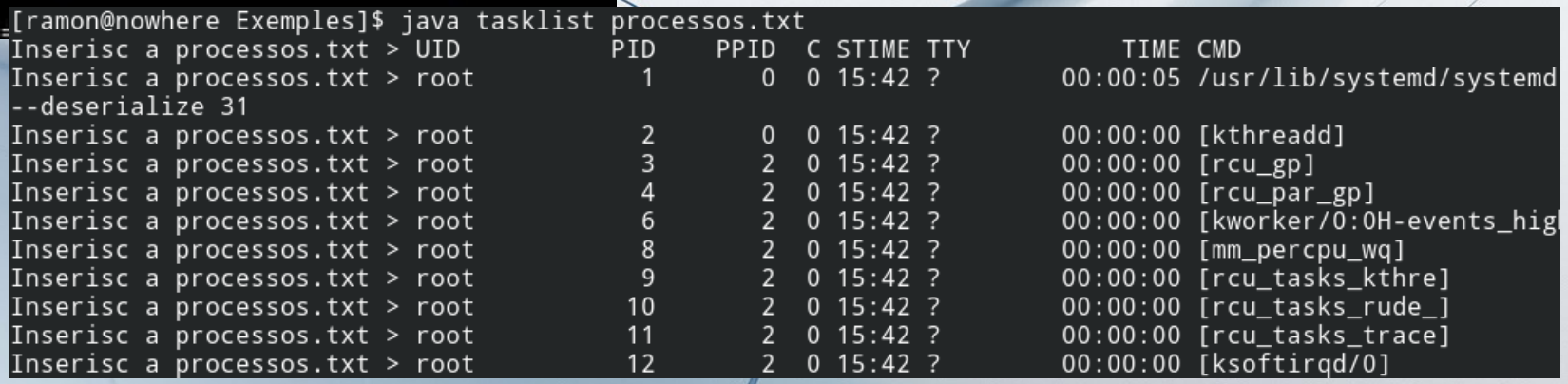
```
[ramon@nowhere Exemples]$ javac Exemple3.java
[ramon@nowhere Exemples]$ java Exemple3 fic.txt
Inserisc en fic.txt > 1."Hola_Món!!"
Inserisc en fic.txt > 2."Hola_Món!!"
Inserisc en fic.txt > 3."Hola_Món!!"
Inserisc en fic.txt > 4."Hola_Món!!"
Inserisc en fic.txt > 5."Hola_Món!!"
Valor d'eixida 0
[ramon@nowhere Exemples]$ cat fic.txt
1."Hola_Món!!"
2."Hola_Món!!"
3."Hola_Món!!"
4."Hola_Món!!"
5."Hola_Món!!"
```

Activitat T2S1P2tasklistGuarda

Realitza un programa que genere un procés que guarda en un fitxer de nom processos.txt la informació dels processos en execució:

Linux: Per a veure els processos executa $ ps -ef
Windows: Per a veure els processos executa C:\>tasklist /v





Process. getOutputStream

Anem a vore com introduir ordres al subprocés, és a dir, com escriure en el flux d'entrada.

Process. getOutputStream

Process també té el mètode **getOutputSteam()** que permet escriure en el flux d'entrada del procés.

És a dir, passar-li els valors que ens sol·licitarà el procés.

Exemple:

- A Windows executa "date"
- A Linux executa "passwd"

```
[ramon@nowhere Exemples]$ passwd
Cambiando la contraseña del usuario ramon.
Contraseña actual:
```

```
C:\pss>date
The current date is: 11/10/2013
Enter the new date: (dd-mm-yy) 10/10/2013_
```

Exemple5. Windows/Linux

Anem a implementar l'Exemple5:

- A Windows executarà "date" i li passarà els valors 2013.10.10.
- A Linux executarem "passwd" per a canviar la contrasenya.

Primera part igual que tots els exemples. Només canvia el comando

Exemple5.java

```
1  import java.io.*;
2  public class Exemple5 {
3      public static void main(String[] args){
4      Runtime r=Runtime.getRuntime();
5      //Linux. Canvi de contrasenya
6      String comando="passwd"; //comando a executar
7      //Windows. Canvi de data
8      //String comando="CMD /C DATE"; //comando a executar
9      Process p=null;
```

Exemple5. Windows/Linux

```
try{
    p=r.exec(comando);
    //escriptura - envie entrada a "date"
    OutputStream os=p.getOutputStream();
    //Linux
    os.write("contrasenyaactual\n".getBytes()); //Contrasenya actual
    os.flush(); //buide buffer
    os.write("contrasenyanova\n".getBytes()); //Contrasenya nova
    os.flush(); //buide buffer
    os.write("contrasenyanova\n".getBytes()); //Repetir contrasenya nova
    os.flush(); //buide buffer
    //Windows
    /*os.write("10-10-13\n".getBytes());
    os.flush(); //buide buffer*/

    //lectura - obté l'eixida de "date"
    InputStream is=p.getInputStream();
    BufferedReader br=new BufferedReader(new InputStreamReader (is));
    String linea;
    while ((linea=br.readLine())!=null)//llig una línia
        System.out.println(linea);
    br.close();
    } catch(Exception e) {
        System.out.println ("Error en "+comando);
        e.printStackTrace();
        }
```

Afegim escriptura, per a l'enviament de dades al procés p

A Linux cal ficar la contrasenya vella, després la nova i repetir la nova una altra vegada

lectura igual que en exemples anteriors

Exemple5. Windows/Linux

```
37
38          //comprovació d'error: 0 bé - 1 malament
39          int exitVal;
40          try {
41              exitVal=p.waitFor();
42              System.out.println("Valor d'eixida "+exitVal);
43              } catch(InterruptedException e){
44                  e.printStackTrace();
45                  }
46
47          }//fi main
48      }//fi exemple5
```

comprovació d'errors igual que en altres exemples

Exemple5. Execució

```
C:\Users\david\Desktop\procesos_java1>java Ejemplo5
The current date is: 27/09/2016
Enter the new date: (dd-mm-yy) 10-10-13
```

```
[ramon@nowhere Exemples]$ javac Exemple5.java
[ramon@nowhere Exemples]$ java Exemple5
Cambiando la contraseña del usuario ramon.
Cambiando la contraseña del usuario ramon.
passwd: todos los tokens de autenticación se actualizaron exitosamente.
Valor d'eixida 0
```

Process. getOutputStream

Realitzem un programa en Java que llija una cadena de l'entrada estàndard: ExempleLectura.java

ExempleLectura.java

```
1   import java.io.*;
2   public class ExempleLectura{
3       public static void main (String[] args){
4       InputStreamReader in=new InputStreamReader(System.in);
5       BufferedReader br = new BufferedReader (in);
6       String texto;
7
8       try{
9           System.out.println("Introdueix una cadena...");
10          texto=br.readLine();
11          System.out.println("Cadena escrita: "+texto);
12          in.close();
13          }catch (Exception e) {e.printStackTrace();}
14
15      }//fi main
16  }//fi ExempleLectura
```

Process. getOutputStream

```
[ramon@nowhere Exemples]$ javac ExempleLectura.java
[ramon@nowhere Exemples]$ java ExempleLectura
Introdueix una cadena...
Hola Ramon. Mola PSP!
Cadena escrita: Hola Ramon. Mola PSP!
```

Activitat T2S1P3CridaLectura

Realitza un programa T2S1P3CridaLectura que cride al programa T2S1ExempleLectura.java y li envie la cadena "Hola món". També haurà d'arreplegar el flux d'eixida i d'error.